

Crafting Automated Tests for your PHP Applications

March, 20th 2025 - Dutch PHP Conference - Amsterdam, The Netherlands

kodero



Hugo HAMON

Consultant PHP / Symfony @ **KODERO**

Symfony Certified Expert Developer

@hhamon / hello@kodero.fr



Automated Tests at the Heart of Software Development

"Code without tests is bad code. It doesn't matter how well-written it is; it doesn't matter how pretty or object-oriented or well-encapsulated it is."

"With tests, we can change the behavior of our code quickly and verifiably. Without them, we really don't know if our code is getting better or worse."

— Michael C. Feathers



https://www.pmi.org/learning/library/legacy-it-systems-upgrades-11443

Reasons for testing your apps

Validating a new added feature

Fixing a persistent or business impacting bug

Improve or simplify the overall “shape” of your code

Detecting and optimizing performance issues

Maintain your application for the years to come

Replacing an old piece of software for a modern one

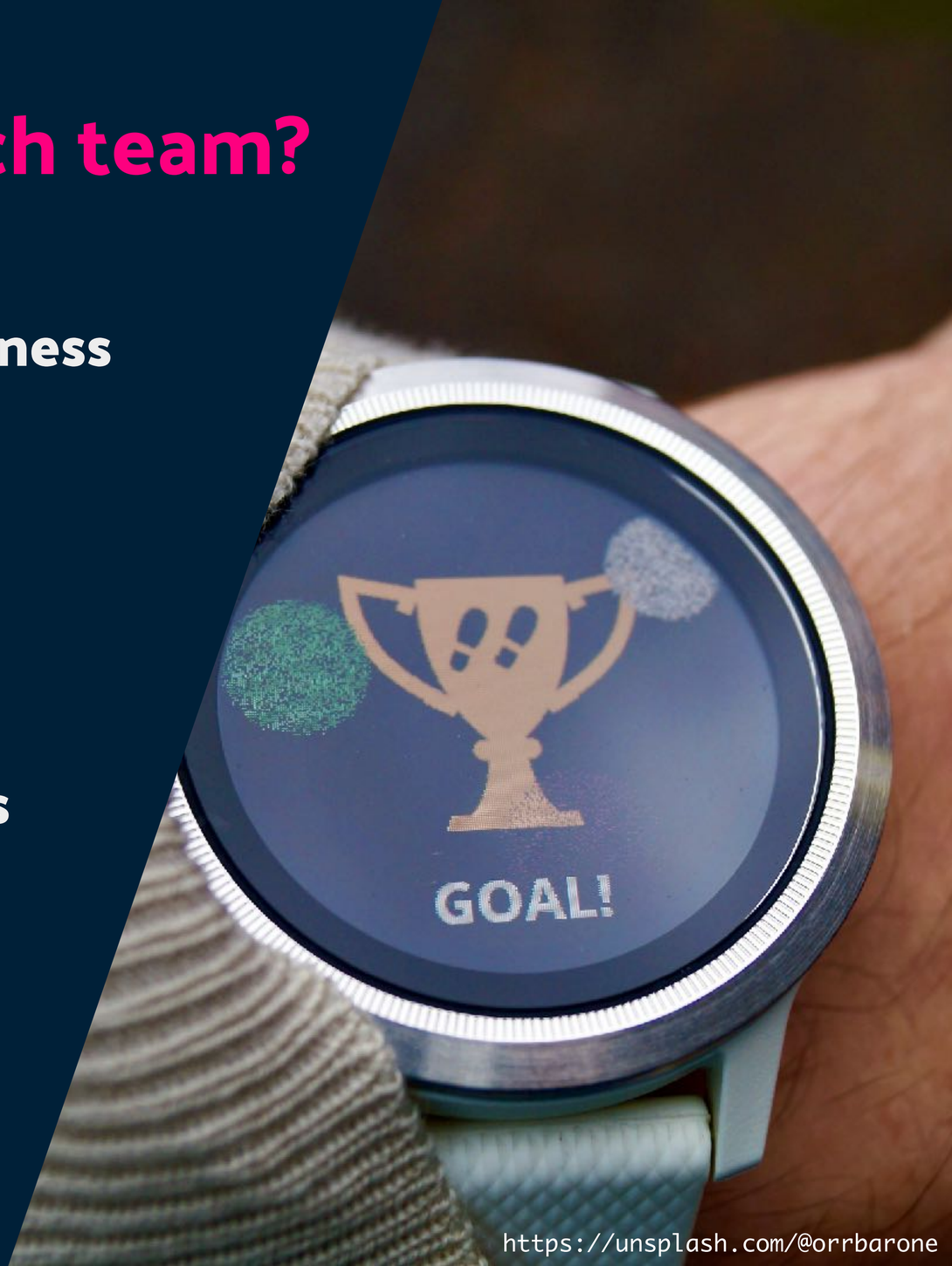
Easing infrastructure & dependencies upgrades

…



What are the benefits for the tech team?

- Ensuring the code works and fulfils the business requirements or tech specifications
- Finding potential bugs before they occur in production
- Lowering the risk of introducing regressions when changing the code
- Keeping focusing on bringing value to the business and end users on the long run



https://unsplash.com/@orrbarone

Different types of tests

- Others
- E2E Tests
- Application Tests
- Integration Tests
- Unit Tests



https://unsplash.com/@imchenyf

PHP community libraries for testing and code quality monitoring

Do not reinvent the wheel by reusing code from others



Useful Libraries

- beberlei/assert
- behat/behat
- dama/doctrine-test-bundle
- fakerphp/faker
- friends-of-behat/symfony-extension
- league/flysystem
- monolog/monolog
- mtdowling/jmespath.php
- nesbot/carbon
- phpstan/phpstan
- phpstan/phpstan-doctrine
- phpstan/phpstan-phpunit
- phpstan/phpstan-symfony
- phpunit/phpunit
- symfony/browser-kit
- symfony/clock
- symfony/css-selector
- symfony/panther
- symfony/string
- zenstruck/assert
- zenstruck/browser
- zenstruck/foundry
- …



Unit Testing

Unit Testing

“ In software development, unit testing is a procedure enabling to verify the correct behavior of a single or small piece of software (called a « unit » ou « module »). ”

– Wikipedia



https://unsplash.com/@alexcioaba

F.I.R.S.T Rule

⚡ Fast

📦 Isolated

🔁 Repeatable

☑️ Self Validating

⏰ Timely - Thorough



TDD ⊙ Test Driven Development

Write unit test first

Code the stub

Run the unit test and see it fails

Code the implementation

Run the unit test again & see it passes

Rinse and repeat...



A.A.A Pattern

Arrange

Prepend the unit test requirements such as resetting the environnement, instantiating the SUT (and its dependencies), loading some fixture data, etc. It includes writing an explicit and self explanatory test method.

Act

Execute the sequence of methods on the SUT.

Assert

Perform some assertions on the SUT to ensure the outcome matches the specifications of the test.



https://unsplash.com/@nathan_cima

Model Class

```php
class Loan
{
    // ... getter + setter methods

    public function getInstallmentsValue(): float
    {
        $borrowedValue = $this->getBorrowedValue();
        $installments = $this->getTotalInstallments();
        $fee = $this->getMonthlyFee();

        return ($borrowedValue / $installments) * (1 + ($fee / 100));
    }
}
```



PHPUnit

```php
final class LoanTest extends TestCase
{
    public function testGetLoanInstallmentValue(): void
    {
        $customer = new Customer();

        $loan = new Loan();
        $loan->setCustomer($customer);
        $loan->setBorrowedValue(10_000);
        $loan->setTotalInstallments(60);
        $loan->setMonthlyFee(3.25);

        self::assertSame($customer, $loan->getCustomer());
        self::assertSame(10_000, $loan->getBorrowedValue());
        self::assertSame(3.25, $loan->getMonthlyFee());
        self::assertSame(60, $loan->getTotalInstallments());

        // Expecting a decimal amount with cents
        self::assertSame(172.08, $loan->getInstallmentsValue());
    }
}
```



PHPUnit - Test Suite

```
 ✘ hhamon@hhamon-kodero-mbp16 ~/Code/modernize/money-manager php bin/phpunit --debug
PHPUnit 8.5.39 by Sebastian Bergmann and contributors.

Testing Project Test Suite
Test 'App\Tests\Entity\LoanTest::testGetLoanInstallmentValue' started
Test 'App\Tests\Entity\LoanTest::testGetLoanInstallmentValue' ended


Time: 33 ms, Memory: 4.00 MB

There was 1 failure:

1) App\Tests\Entity\LoanTest::testGetLoanInstallmentValue
Failed asserting that 172.08333333333331 is identical to 172.08.

/Users/hhamon/Code/modernize/money-manager/tests/Entity/LoanTest.php:29

FAILURES!
Tests: 1, Assertions: 5, Failures: 1.
 ✘ hhamon@hhamon-kodero-mbp16 ~/Code/modernize/money-manager
```



Model Class

```php
class Loan
{
    // ...

    public function getInstallmentsValue(): float
    {
        $borrowedValue = $this->getBorrowedValue();
        $installments = $this->getTotalInstallments();
        $ratio = 1 + $this->getMonthlyFee() * 0.01;

        $amount = ($borrowedValue / $installments) * $ratio;

        return round($amount, 2, PHP_ROUND_HALF_UP);
    }
}
```



PHPUnit - Test Suite

```
 hhamon@hhamon-kodero-mbp16  ~/Code/modernize/money-manager  php bin/phpunit --debug
PHPUnit 8.5.39 by Sebastian Bergmann and contributors.

Testing Project Test Suite
Test 'App\Tests\Entity\LoanTest::testGetLoanInstallmentValue' started
Test 'App\Tests\Entity\LoanTest::testGetLoanInstallmentValue' ended


Time: 33 ms, Memory: 4.00 MB

OK (1 test, 5 assertions)
 hhamon@hhamon-kodero-mbp16  ~/Code/modernize/money-manager
```

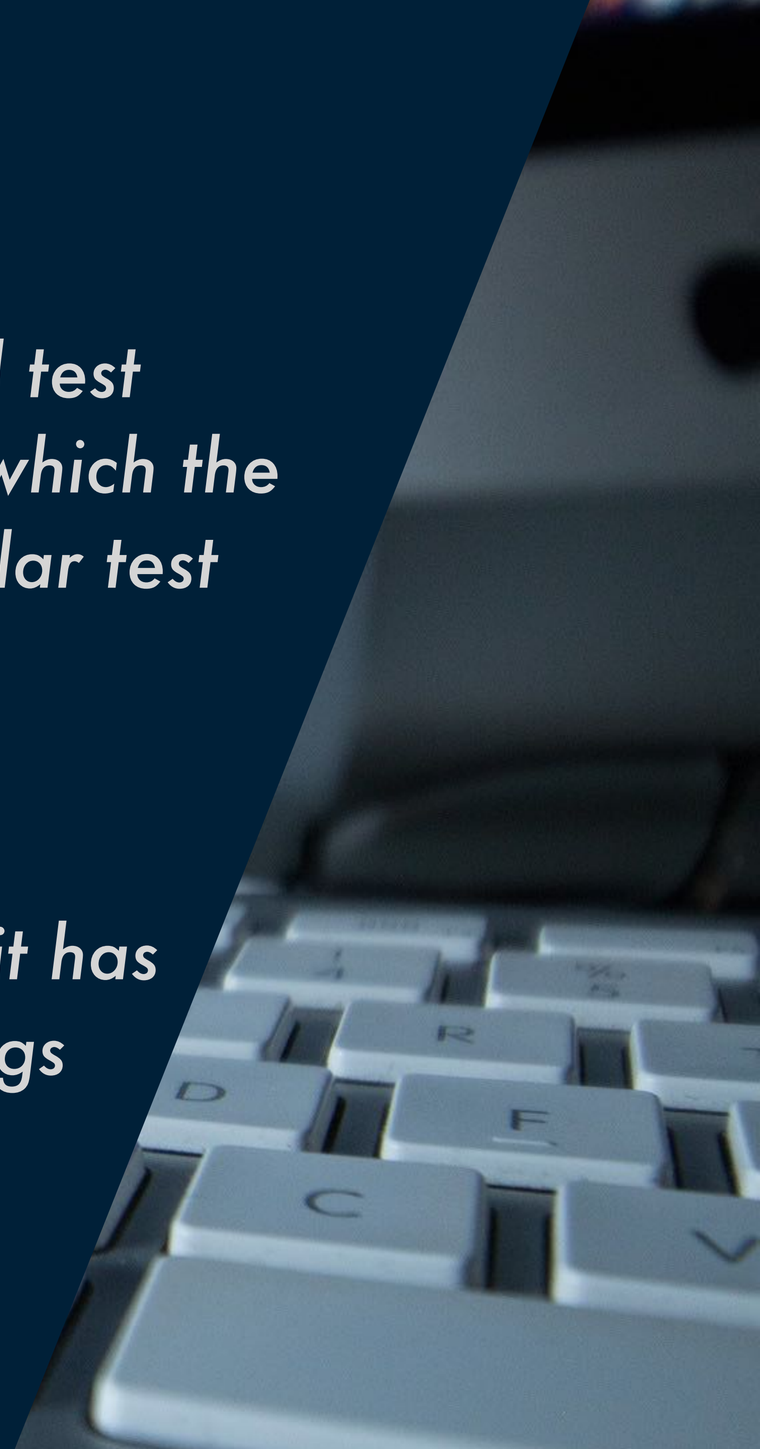


Code Coverage

" In software engineering, code coverage, also called test coverage, is a percentage measure of the degree to which the source code of a program is executed when a particular test suite is run. "

" A program with high code coverage has more of its source code executed during testing, which suggests it has a lower chance of containing undetected software bugs compared to a program with low code coverage. "

– Wikipedia



PHPUnit

```xml
<?xml version="1.0" encoding="UTF-8"?>
<phpunit xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" ... >

    ...

    <!-- Also use pathCoverage="true" if driver supports path code coverage -->
    <coverage processUncoveredFiles="true" cacheDirectory="build/phpunit/cache">
        <include>
            <directory suffix=".php">src</directory>
        </include>
        <report>
            <html outputDirectory="build/coverage/html"/>
            <text outputFile="build/coverage/coverage.txt"/>
            <clover outputFile="build/coverage/clover.xml"/>
        </report>
    </coverage>

</phpunit>
```




```
$ phpdbg -qrr vendor/bin/phpunit --coverage-text
```

/Users/hhamon/Code/confoo-2025/src / (Dashboard)

	Code Coverage								
	Lines			Functions and Methods			Classes and Traits		
Total		0.21%	1 / 468		0.91%	1 / 110		4.35%	1 / 23
Api		0.00%	0 / 32		0.00%	0 / 11		0.00%	0 / 1
Command		0.00%	0 / 13		0.00%	0 / 3		0.00%	0 / 1
Controller		0.00%	0 / 150		0.00%	0 / 8		0.00%	0 / 7
DataFixtures		0.00%	0 / 94		0.00%	0 / 4		0.00%	0 / 1
Entity		0.00%	0 / 95		0.00%	0 / 61		0.00%	0 / 6
Form		0.00%	0 / 10		0.00%	0 / 2		0.00%	0 / 1
Model		0.00%	0 / 12		0.00%	0 / 9		0.00%	0 / 1
Repository		2.56%	1 / 39		12.50%	1 / 8		25.00%	1 / 4
Service		0.00%	0 / 23		0.00%	0 / 4		0.00%	0 / 1
Kernel.php		n/a	0 / 0		n/a	0 / 0		n/a	0 / 0

Legend

Low: 0% to 50% **Medium**: 50% to 90% **High**: 90% to 100%

Generated by php-code-coverage 9.2.32 using PHPDBG 8.4.4 and PHPUnit 9.6.22 at Mon Feb 24 9:26:58 UTC 2025.

PHPUnit



Integration Testing

Integration Testing

" Integration testing, also called integration and testing, abbreviated I&T, is a form of software testing in which multiple parts of a software system are tested as a group. "

– Wikipedia



https://unsplash.com/@imchenyf

Database Seeding

Sometimes, integration tests may run code that interacts with the database.

Therefore, it's important to dedicate a database exclusively for the "test" environment.

The system must be able to fully truncate the database data and reload fresh data fixtures between each test scenario.



```php
$faker = \Faker\Factory::create();

for ($i = 0; $i < 5; $i++) {
    $address = new Address(
        $faker->streetAddress(),
        $faker->postcode(),
        $faker->city(),
        $faker->country(),
    );

    $bio = $faker->paragraphs($faker->numberBetween(2, 6), asText: true);

    $person = new Person($faker->firstName(), $faker->lastName());
    $person->setAddress($address)
    $person->setIntroduction($faker->text(maxNbChars: 164));
    $person->setBiography($bio);
}
```

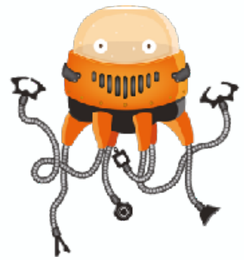
https://github.com/FakerPHP/Faker



```
App\Entity\User:
    user{1..10}:
        username: '<username()>'
        fullname: '<firstName()> <lastName()>'
        birthDate: '<date_create()>'
        email: '<email()>'
        favoriteNumber: '50%? <numberBetween(1, 200)>'

App\Entity\Group:
    group1:
        name: Admins
        owner: '@user1'
        members: '<numberBetween(1, 10)>x @user*'
        created: '<dateTimeBetween("-200 days", "now")>'
        updated: '<dateTimeBetween($created, "now")>'
```

https://github.com/nelmio/alice



```php
use Symfony\Bundle\FrameworkBundle\Test\WebTestCase;
use Zenstruck\Foundry\Test\Factories;
use Zenstruck\Foundry\Test\ResetDatabase;

final class PostPublicationTest extends KernelTestCase
{
    use ResetDatabase, Factories;

    public function testUnpublishedPost(): void
    {
        $post = PostFactory::new()
            ->published()
            ->create([
                'slug' => 'superman-seen-in-metropolis',
            ]);

        // ...
    }
}
```

https://github.com/zenstruck/foundry



```php
use Symfony\Bundle\FrameworkBundle\Test\KernelTestCase;
use Zenstruck\Foundry\Test\Factories;
use Zenstruck\Foundry\Test\ResetDatabase;
use Zenstruck\Mailer\Test\InteractsWithMailer;

final class CancelAppointmentTest extends KernelTestCase
{
    use Factories;
    use InteractsWithMailer;
    use ResetDatabase;

    protected function setUp(): void
    {
        parent::setUp();

        self::bootKernel();
    }

    private function getCancelAppointmentService(): CancelAppointment
    {
        return self::getContainer()->get(CancelAppointment::class);
    }

    // …
}
```

Integration tests are setup thanks to the base Symfony KernelTestCase class

```php
use Symfony\Bundle\FrameworkBundle\Test\KernelTestCase;
use Zenstruck\Foundry\Test\Factories;
use Zenstruck\Foundry\Test\ResetDatabase;
use Zenstruck\Mailer\Test\InteractsWithMailer;

final class CancelAppointmentTest extends KernelTestCase
{
    // …

    public function testScheduledUpcomingAppointmentCanBeCancelled(): void
    {
        // Arrange: seed the database with a scheduled upcoming appointment
        $appointment = MedicalAppointmentFactory::new([])
            ->tomorrow('10:00', '11:30')
            ->create([
                'firstName' => 'John',
                'lastName' => 'Smith',
                'email' => 'user@example.com',
            ]);

        // Act: cancel the appointment
        $this->getCancelAppointmentService()->cancel($appointment->_real(), 'I found one earlier.');

        // Assert: check the appointment is cancelled in the database
        MedicalAppointmentFactory::assert()->count(1, [
            'id' => $appointment->getId(),
            'cancellationReason' => 'I found one earlier.',
        ]);

        self::assertTrue($appointment->isCancelled());
        self::assertInstanceOf(DateTimeInterface::class, $appointment->getCancelledAt());
        self::assertSame('I found one earlier.', $appointment->getCancellationReason());

        // Assert: check the confirmation email has been sent
        $this->mailer()
            ->assertSentEmailCount(1)
            ->assertEmailSentTo('user@example.com', 'Your appointment has been cancelled');
    }
}
```

```
PHPUnit 9.6.22 by Sebastian Bergmann and contributors.

Warning:       No code coverage driver available

Testing
.                                                                   1 / 1 (100%)

Time: 00:00.595, Memory: 36.50 MB

OK (1 test, 6 assertions)
```

```
$ symfony php vendor/bin/phpunit --filter testScheduledUpcomingAppointmentCanBeCancelled
```



Application Tests

Application Tests

Application tests are a kind of blackbox testing approach whose purpose is mainly to execute all layers of the applications ; from the request to the response or from the input to the output.



https://unsplash.com/@markusspiske

Pros

- ★ Ideal for testing the external behavior
- ★ Adapted for “smoke testing”
- ★ All applications layers are being called
- ★ Validating workflows

Cons

- ★ More complex to setup
- ★ Slowness (networks, IO, 3rd party, etc.)
- ★ Mostly require a tight framework integration
- ★ Cannot be used for testing JavaScript heavy apps

```php
final class PostAppointmentCancellationControllerTest extends WebTestCase
{
    use Factories;
    use InteractsWithMailer;
    use ResetDatabase;

    public function testCannotCancelPastAppointment(): void
    {
        // Arrange: populate the database with the needed fixtures
        $appointment = MedicalAppointmentFactory::new()
            ->scheduled('E4N6ST')
            ->yesterdayAt('10:00', '11:30')
            ->forPatient('John', 'Smith')
            ->create();

        // Arrange: setup the runtime environment (aka kernel)
        self::ensureKernelShutdown();

        $client = static::createClient();
        $client->setServerParameter('HTTP_ACCEPT', 'application/json');

        // ...
    }
}
```

```php
final class PostAppointmentCancellationControllerTest extends WebTestCase
{
    public function testCannotCancelPastAppointment(): void
    {
        // ...

        // Act: call the API endpoint with the client
        $client->request(
            method: 'POST',
            uri: \sprintf('/api/appointments/%s/cancellation', $appointment->getId()),
            content: \json_encode(
                value: [
                    'referenceNumber' => 'E4N6ST',
                    'lastName' => 'SMITH',
                    'reason' => 'I booked another one earlier.',
                ],
                flags: \JSON_THROW_ON_ERROR,
            ),
        );

        // ...
    }
}
```

```php
final class PostAppointmentCancellationControllerTest extends WebTestCase
{
    public function testCannotCancelPastAppointment(): void
    {
        // ...

        // Assert: ensure the API HTTP endpoint returned a 422 HTTP response
        self::assertResponseIsUnprocessable();
        self::assertResponseHeaderSame('Content-Type', 'application/json');

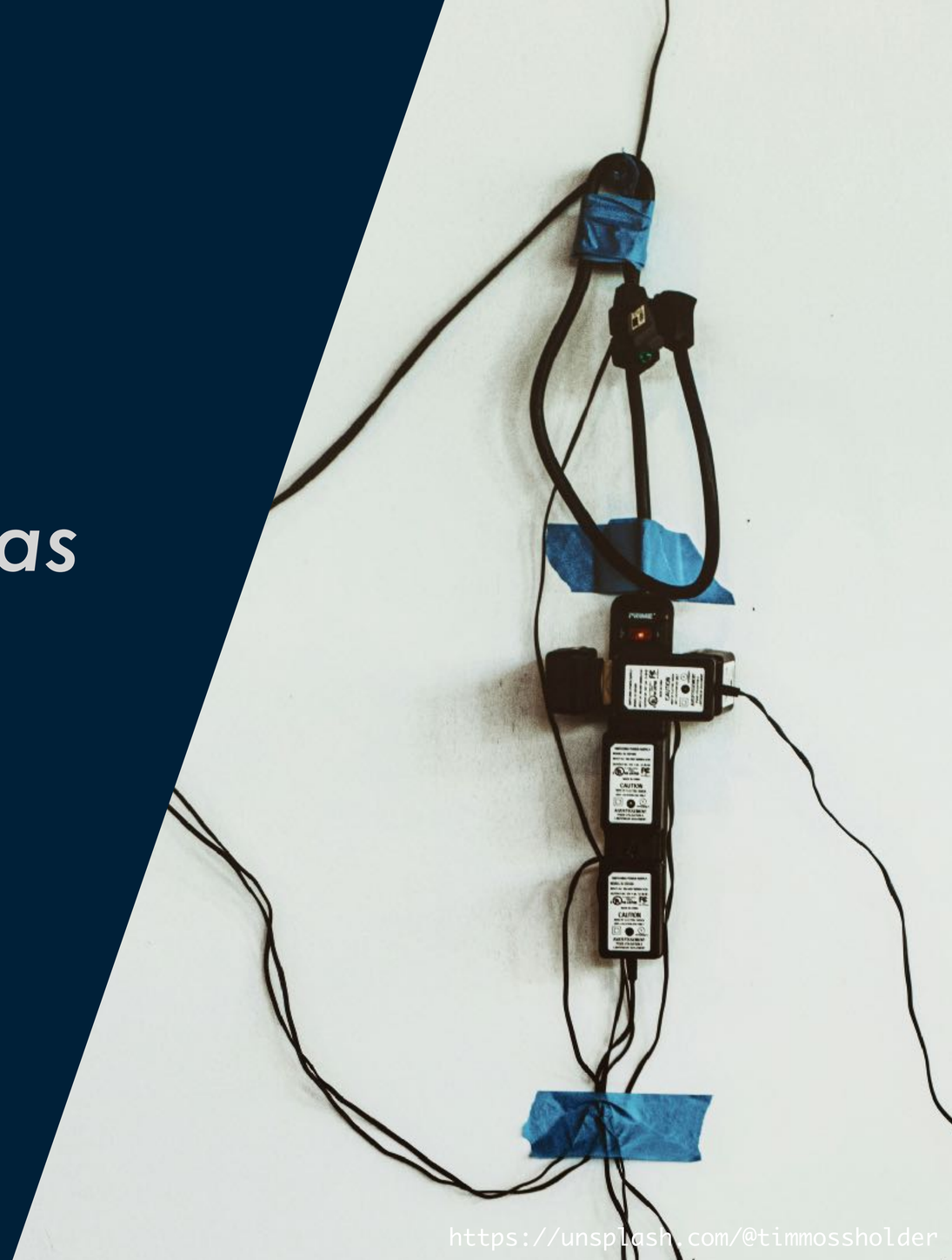
        self::assertStringContainsString(
            'Appointment is no longer cancellable.',
            $client->getResponse()->getContent(),
        );
    }
}
```



End-to-End Testing

“End-to-End” Testing

“ The end-to-end (or E2E) tests enable to validate the system as a whole by taking all its application layers into account. ”



Pros

- ★ Ideal for testing the external behavior
- ★ Adapted for “smoke testing”
- ★ All applications layers are being called
- ★ Validating UI interactions and workflows

Cons

- ★ More complex to setup
- ★ Slowness (networks, IO, 3rd party, etc.)
- ★ More difficult to write

```php
use Symfony\Component\Panther\Client;

require __DIR__.'/vendor/autoload.php';

$client = Client::createChromeClient();

// Interact with the UI elements
$client->request('GET', 'https://api-platform.com');
$client->clickLink('Getting started');

// Wait for an element to be present in the DOM (even if hidden)
$crawler = $client->waitFor('#installing-the-framework');

// Alternatively, wait for an element to be visible
$crawler = $client->waitForVisibility('#installing-the-framework');

echo $crawler->filter('#installing-the-framework')->text();

// Yeah, screenshot!
$client->takeScreenshot('screen.png');
```



Panther Library

```
Feature:
    In order to have a localized experience,
    users must be able to switch between supported languages.

    Scenario: A user is able to switch to English language.
        Given a guest user visits the French speaking website
        When they switch language to "english"
        Then they see "Create your account" as page title

    Scenario: A user is able to switch to French language.
        Given a guest user visits the English speaking website
        When they switch language to "français"
        Then they see "Créez votre compte" as page title
```



Behat + Gherkin

```php
final class SwitchLanguageContext implements Context
{
    private Client $client;
    private Crawler $crawler;

    public function __construct()
    {
        $this->client = Client::createChromeClient();
    }

    #[Given('a guest user visits the English speaking website')]
    public function guestUserVisitsTheEnglishSpeakingWebsite(): void
    {
        $this->crawler = $this->client->request('GET', 'https://test.website.com');
    }

    #[When('they switch language to :language')]
    public function theySwitchLanguageTo(string $language): void
    {
        $this->crawler = $this->client->clickLink($language);
    }

    #[Then('they see :title as page title')]
    public function theySeeAsPageTitle(string $title): void
    {
        if ($title !== $this->crawler->filter('title')->text()) {
            throw new \Exception('Title mismatch');
        }
    }
}
```

Behat + Gherkin + Panther





Thank You!

kodero

@hhamon / hello@kodero.fr